

How to produce forecast stats and plots using stats.py and plots_corcmp.py/plots_syscmp.py

Michelle Frazer, updated 20-Jul-2026

Table of Contents

STATS.....	2
1. How stats.py works	2
a. Input.....	2
b. Output	2
c. Statistics	3
d. Regional stats: masks.....	4
e. Global stats: windows.....	5
f. Calculated variables	6
2. How to run stats.py	6
a. Configuration.....	6
b. Use via command line	8
c. Use via stats.run	9
PLOTS	12
1. How plots_corcmp.py and plots_syscmp.py work.....	12
a. Input.....	12
b. Output.....	12
c. Confidence intervals	13
d. Example plots	14
Figure 1: example dieoff curve plot.....	14
Figure 2: example zonal mean comparison plot (linear 100-mb)	15
Figure 3: example RMS decomposition montage plot	15
Figure 4: example regional montage plot (log 10-mb)	16
Figure 5: example overview stats montage plot (linear 10-mb; land-only)	16
Figure 6: example global map individual static plot.....	17
Figure 7: example global zonal mean comparison static plot (log 10-mb)	17
2. How to run plots_corcmp.py and plots_syscmp.py	17
a. Configuration.....	17
b. Use via command line	18
c. Use via plots.run	19
d. Usage of plots_syscmp.py: need for cartopy	20
e. Syncing with Polar	20

STATS

The stats.py script calculates stats (RMS, ACORR, etc.) for input forecast, analysis, and climatology data. It functionally replaces stats.x (as well as the stats calculations included in the plotsys and syscmp plotting routines).

1. How stats.py works

a. Input

The input files should be netcdf and should have a time coordinate named “time” or “valid_time”. Each forecast initialization date should have its own file or files, depending on whether the files are multi- or single-lead. Single-lead files have a time coordinate of length 1, while multi-lead files have a time coordinate of longer length. There can be a single analysis file for each datetime to be analyzed, or analysis files containing multiple timesteps.

Climatology files should contain monthly data, with one for each analysis time (00z, 06z, etc) stats will be computed for. The climatology files must have a time dimension of length 12. For each analysis time, stats.py will calculate a simple distance-weighted climatology. For example, the climatology for June 25 12z would be the sum of 68.85% June and 31.15% July (as it is 9.5 days from the center of June, but 23 days from the center of July).

b. Output

The script will output several netcdf files (as opposed to the .ctl/.data GrADS files produced by stats.x) depending on the options used. When processing new datasets and not in single-forecast mode, the processed forecast, analysis, and climatology will be saved as respective netcdf files (allowing for easy visualization to trouble-shoot input data).

Two output stats netcdf files can be produced, one for regional stats and one for global stats. Both will contain multiple stats for each processed variable, for example, H_acorr and H_rms. If not in single-forecast mode, these files will also contain the same variables averaged over init_date (for example, H_acorr_avg and H_rms_avg) and global files will also contain global averages over both init_date and lat/lon (for example, H_acorr_glo and H_rms_glo. In a regional stats file, a 3D variable like H would have dimensions of init_date (datetime), lead (hours), lev (hPa), and region (string abbreviation like “NHE”). If a global stats file, the region dimension would be replaced by lat and lon.

When running regional stats, input data for scorecards is also produced as .txt files for each forecast (identical to those produced by stats.x). These files are formatted like the following example lines:

```

count|date|domain_name|east|expver|forecast|level|levtype|north
|variable|source|south|statistic|step|type|value|verify|west
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|cor|0|fc|1.000000|gmao|-180.0000|
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|rms|0|fc|0.0000000|gmao|-180.0000|
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|rms_ran|0|fc|0.0000000|gmao|-180.0000|
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|rms_bar|0|fc|0.0000000|gmao|-180.0000|
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|rms_dis|0|fc|0.0000000|gmao|-180.0000|
0.0|2025080100|global|180.0000|f5295fp|gmao|1000.000|pl|90.00000
|h|gmao|-90.00000|rms_dsp|0|fc|-7.382045E-04|gmao|-180.0000|

```

Note there are some quirks in these lines as holdovers from stats.x. Some columns always have the same values: “count” always shows 0.0, “levtype” always shows “pl”, and “type” always shows “fc”. Both “forecast” and “source” columns show the value of the expver parameter from the yaml configuration file, while “expver” shows the model name (the model parameter from the yaml); see the “Configuration” section below.

The main information columns are “domain_name” (region), level, variable, statistic, step (lead time in hours) and value. For consistency with previous scorecards, specific humidity (q and q2m) is given in base units of kg/kg (while the stats files use g/kg).

c. Statistics

Both regional and global stats modes calculate and output anomaly correlation (acorr), root mean squared error (rms), amplitude component of RMS Error (rms_amp), bias component of RMS Error (rms_bar), and phase component of RMS Error (rms_phz). Regional stats also include (for scorecard input data database purposes): dissipation component of RMS Error (rms_dis), dispersion component of RMS Error (rms_dsp), and random component of RMS Error (rms_ran). Global stats add for plotting purposes: analysis (a), forecast (f), and forecast minus climatology (f_c) values as well as mean error (me). In terms of units, rms, rms_amp, rms_bar, rms_phz, and rms_ran take the units of the underlying variable, while mse, rms_dis, rms_dsp use the square of the underlying variable’s units, and acorr is unitless.

For a more through explanation of the RMS decomposition components, see the RMS Decomposition Definitions (Animated gif) and A Review of Anomaly Correlation and RMS Error (pdf) files linked at the bottom of the “Forecast Statistics” page for each experiment on the GMAO GEOS-5 Model Development page.

The equations used in stats.py are as follows (and are identical to the equations used in stats.x). For regional stats, weighted (by cell area) means are taken over all non-NaN lat/lon cells in the given region; for global stats, the means are for the window centered on the lat/lon cell being calculated for (see “Global stats: windows” section below).

Means/deviations:

```
fmean = mean(f)
amean = mean(a)
cmean = mean(c)
fstar = f - fmean
astar = a - amean
cstar = c - cmean
```

Variances:

```
fvar = mean((fstar - cstar) * (fstar - cstar))
avar = mean((astar - cstar) * (astar - cstar))
cvar = mean((fstar - cstar) * (astar - cstar))
varf = mean(fstar * fstar)
vara = mean(astar * astar)
varc = mean(fstar * astar)
stdf = varf**(1/2)
stda = vara**(1/2)
```

Mean error, mean squared error, and root mean squared error:

```
me = mean(f - a)
mse = mean((f - a)**2)
rms = mse**(1/2)
```

Anomaly correlation:

```
acorr = cvar / (fvar**(1/2) * avar**(1/2))
```

Root bias error:

```
rms_bar = abs(fmean - amean)
```

Dissipation and root amplitude error:

```
rms_dis = (stdf - stda)**2 + (rms_bar)**2
rms_amp = abs(stdf - stda)
```

Dispersion and root phase error:

```
rho = varc / (stdf * stda)
rms_dsp = 2*(1-rho)*stdf*stda
rms_phz = rms_dsp**(1/2)
```

Root random error:

```
rms_ran = (mean((fstar - astar)**2))**(1/2)
```

With these definitions:

```
mse = rms_bar**2 + rms_amp**2 + rms_phz**2 = rms_bar**2 + rms_ran**2
```

d. Regional stats: masks

Region masks used for stats can be defined by lat/lon coordinates, drawn from a netcdf mask file, or created as the intersection of other regions.

The following regions are defined by simple lat/lon coordinate bounds (N, S, E, W):

GLO: [90.0, -90.0, 180.0, -180.0]
NHE: [80.0, 20.0, 180.0, -180.0]
TRO: [20.0, -20.0, 180.0, -180.0]
SHE: [-20.0, -80.0, 180.0, -180.0]
NWQ: [90.0, 0.0, 0.0, -180.0]
NEQ: [90.0, 0.0, 180.0, 0.0]
SWQ: [0.0, -90.0, 0.0, -180.0]
SEQ: [0.0, -90.0, 180.0, 0.0]
NAM: [60.0, 20.0, -60.0, -140.0]
EUR: [60.0, 30.0, 30.0, -10.0]
NPO: [90.0, 60.0, 180.0, -180.0]
SPO: [-60.0, -90.0, 180.0, -180.0]
XPO: [60.0, -60.0, 180.0, -180.0]

LND (land, produced by cartopy at 0.5-deg resolution) and CUS (continental US, from NCEP) regions come from NetCDF mask files.

Land-only versions of NHE, TRO, and SHE come from intersections:

NHL: [NHE, LND]
TRL: [TRO, LND]
SHL: [SHE, LND]

e. Global stats: windows

Calculating stats for every lat/lon point is messy because the calculations require spatial means. In theory, temporal means are possible, but taking means of $f/a/c$ to produce star values (for example, $f_{star} = f - f_{mean}$) over `init_dates` does not provide meaningful interpretation – consider the wave patterns shown in the RMS Decomposition Definitions (Animated gif) referenced above. Unfortunately, the math in the old `plotsys` and `syscmp` plotting routines did take means over `init_dates`, producing results in conflict with those from the `corcmp` plotting routines. The solution utilized by `stats.py` is to have the stats represented by each lat/lon point utilize means over a spatial window centered on the point. The exact size of the window depends on the resolution the stats are being calculated at. The window size is chosen as a “square” with the same odd number of lat/lon points on each edge that is closest to being 15 deg lon across. For example, for the 2 deg lat / 2.5 deg lon resolution used by `stats.x`, the window would have 5 points on each side, coming out to 10 deg lat by 12.5 deg lon. The minimum number of edge points allowed is 5 (for enough points to produce meaningful stats) and the maximum is 21 (out of consideration of processing speed and memory).

When interpreting stats, it is important to keep in mind window (or region) size. For example, calculating stats over a larger window (or region) tends to reduce the Bias component of RMS while increasing the Phase component. Thus, the RMS decomposition is only really useful for

comparing multiple models over the same region (or window size), not for analyzing a given model's breakdown.

f. Calculated variables

The stats.py script allows for calculated variables, that is, variables that are not output by a given source but can be calculated from other available variables. Using metpy, 2-meter dewpoint (D2m) can be calculated from 2-meter specific humidity (Q2m) and surface pressure (PS), or Q2m can be calculate from D2m and PS. For aerosols, 2-meter LOGAOD can be calculated from aerosol optical depth as $\log(\text{AOD}+0.01)$. Third, PM2.5 can be calculated as the sum of SSSMASS25, DUSMASS25, BCSMASS, OCSMASS, SO4SMAS, NISMASS25, and NH4SMAS.

2. How to run stats.py

a. Configuration

Running stats.py requires a yaml configuration file. For templates, see stats_ops_tmpl.yaml for single-forecast (operational) use and stats_dev_tmpl.yaml for multi-forecast (development) use.

length/dates:

Only fcst_length (days) and fcst_interval (hours) are required for single-fcst use. While by default, an initialization hour of 00z is assumed, an alternate value can be provided as init_hour. For multi-forecast use, start_date and end_date are also required (fcst_spacing defaults to 1); expected dates for which data is missing can be excluded as an exclude_dates list.

stats options:

Levels, variables, and regions should all be specified as comma-separated lists in brackets. Order within the list does not matter. Variables are separated by collections: default [H, Q, U, V, T, P, PS], slices [2m and 10m variables], and aerosols. Default variables are separated into 2D and 3D. Regions are only required when running regional stats, as specified by stats_type (or by default in single-forecast mode). In multi-forecast mode, stats.py will look for and validate pre-existing processed fcst/ana/clim datasets with sufficient coverage. The script will always search a directory named "output" within the working directory, but users can specify additional locations to check via dir_loc as a comma-separated list of full paths.

fcst/ana/clim directories/templates:

The fcst_model/ana_model/clim_model parameters are the fcst/ana/clim source names that will be used in all output. These must be purely alphanumeric (no punctuation). The stats.py script will collect input fcst/ana/clim files from the paths made by joining the directory and template entries (wildcards are allowed) and replacing the datetime tags.

Datetime tags must be placed in brackets and begin with “init_” or “valid_”, where init should only be used with forecast paths. Time components -- YYYY (year), MM (month), DD (day), HH (hour), NN (minute), SS (second) – can be combined, for example: {init_YYYYMMDD}, {valid_HHNNSS}, or {init_YYYY-MM-DD_HH}. Note that the script classifies fcst files as single-lead if the fcst template includes any valid time tags. Forward or backward shifts in integer days (d), hours (h), minutes (m), or seconds (s) may be applied to the tags. For example, {init_YYYYMMDD,shift=-3h} translates to a datetime 3 hours prior to the init date being analyzed.

Similarly to datetime tags, forecast hour tags -- {valid_FF} or {valid_FFF} – can be used to specify the forecast lead hour alone as 2- or 3-digit numbers which may represent multiple days. This is helpful since at lead hour 24, {valid_HH} would translate to “00”.

Multiple templates can be listed separated by commas. While templates for default variables are required, templates for slices and aerosols are optional: if not present or left blank they will be assumed to be the same as default. The {expid} tag is optional, but can reduce repetition in the directory/template entries. Note that if a directory or template entry starts with a tag like {expid} or {init_YYYY}, it must be wrapped in quotes (due to issues with yaml file interpretation of the “{” character).

scorecard options:

The model, expver, and verify parameters are only used for the scorecard input files produced. The model parameter will be the forecast model named put in the scorecard database; it may include punctuation. Traditional expver (forecast model source) and verify (verification source) tags used by stats.x are gmao and ecmwf, though the script accepts any tag (ideally no longer than 7 characters).

output grid:

These parameters specify the grid that stats will be produced on: all input datasets will be regridded to this using the xESMF “conservative_normed” option. For example, the traditional grid used by stats.x would be specified by Nlat=91, Nlon=144, delta_lat=2.0, and delta_lon: 2.5. Note an odd Nlat implies half-cells at the poles as is regularly used in the GMAO (the xESMF “conservative_normed” option handles this well).

aliases:

When the script looks for a requested variable in the fcst/ana/clim input files, it will utilize these lists to find a match. The lists are case-insensitive and should not be edited except to make additions.

SBATCH parameters:

These are only used when running stats.py via the stats.run shell script as is discussed below.

b. Use via command line

Running stats.py in single-fcst mode is simple:

```
stats.py [--config CONFIG] [--date DATE] [--save_dir SAVE_DIR]
```

where CONFIG is the yaml configuration file, DATE is the day of the forecast as YYYYMMDD, and SAVE_DIR is the location for the output (stats NetCDF file and scorecard input directory). CONFIG and SAVE_DIR can be specified relative to the working directory or as full paths.

There are many more command line options (all are listed below) which are only relevant for multi-forecast mode. These were largely designed around stats.run to set up a complete workflow with parallelization for optimal efficiency. For these, the “--info_dir” is important as the location where temp and intermediate files and logs will be saved (see “Use via stats.run” section below for more information).

For processing, there are options to check for pre-existing datasets and to process one or all datasets including without stats. There are options to split the processing up into init_date chunks and collections. For statistics, the script can be run to calculate regional, global, or both stats which can also be chunked by init_date. Finally, options exist to facilitate clean merging of chunked or collection-split datasets or stats.

The command line arguments for stats.py are as follows:

```
--config CONFIG Configuration file (default: stats.yaml)
--date DATE      Date YYYYMMDD for single-forecast mode (process all F/A/C
                  datasets and calculate regional stats and scorecard input)
--save_dir SAVE_DIR
                  Directory to save output files (for single forecast mode)
```

Processing options:

```
--check_only      Only check for pre-existing datasets
--fcst            Process only forecast dataset
--ana             Process only analysis dataset
--clim            Process only climatology dataset
--process         Process all datasets (fcst, ana, clim)
--collection TARGET_COLL
                  Process only specific collection (e.g., default, slices,
                  aerosol)
--info_dir INFO_DIR
                  Directory with date range and timestamp
--date_start_idx DATE_START_IDX
                  Starting index for processing date range
--date_end_idx DATE_END_IDX
                  Ending index for processing date range
```

Statistics options:

```
--stats {reg,glo}
                  Run statistics calculation type: reg or glo
--init_start_idx INIT_START_IDX
```

```

        Starting index for init dates (for chunks)
--init_end_idx INIT_END_IDX
        Ending index for init dates (for chunks)
--chunk CHUNK      Forecast chunk number
--chunk_size CHUNK_SIZE
        Forecast chunk size (default: 3)

Merge options:
--merge_collections {fcst,ana,clim}
        Dataset to merge collections (fcst, ana, or clim)
--merge_forecast_chunks
        Merge forecast chunk files
--save_for_coll_merge
        Save merged fcst file to tmp for collection merging
--clean
        Merge chunked statistics files
--type {reg,glo}
        Stats type to merge: reg or glo

```

c. Use via stats.run

The stats.run shell script implements a comprehensive workflow to process stats for a date range efficiently on the NCCS Discover system. It should be run in the command line with only a yaml file as an argument, for example: “./stats.run stats_FP.yaml”. It will look for the stats.py script first in the working directory (WD) then in \$GEOSUTIL/plots_v2/stats_v2.

To begin, stats.run creates a "stats_{fcst}_{ana}_{clim}_{timestamp}" output directory (OD) in WD/output and saves a versioned copy of the referenced yaml configuration file to OD/jobs. The script will check for pre-existing fcst/ana/clim datasets and, if they exist, will validate them against the requested parameters in the yaml. It will then create (in OD/jobs) and submit jobs to the queue with appropriate dependencies (on other jobs) and account, partition, and qos as included in the yaml. The pre-existing dataset check and the job creation/submission will all be summarized in the command line output.

The first set of jobs is fcst/ana/clim dataset creation (as needed). Default, slices, and aerosol collections are processed separately if separate file templates were provided (in the yaml), followed by merge scripts to create unified datasets. Default fcst dataset processing is also chunked followed by its own merge script (as default is the only collection with 3D variables and is therefore more time/memory intensive). Partial files are stored in WD/output/tmp and then removed upon successful merge.

The second set of jobs is stats calculation and runs after all dataset creation jobs are complete. Two separate array jobs run for regional and global stats (if both are requested in the yaml) with the processing chunked by 2-day init_date sections, each followed by its own cleanup jobs to merge the stats chunks (and delete them if successful). The regional stats jobs also produce the txt files with scorecard input (in WD/output/scorecard_input_{fcst}_{ana}_{clim}_{datetime range}).

Thus 5 final netcdf files are produced (if needed) in WD/output, for example:

```
fcst_GEOSFP_20241201-20241231_len8d_int6h_spc1d_91x144.nc4
ana_GEOSFP_20241201-20241231_len8d_int6h_spc1d_91x144.nc4
clim_MERRA2_20241201-20241231_len8d_int6h_spc1d_91x144.nc4
stats_regional_GEOSFP_GEOSFP_MERRA2_20241201-
    20241231_len8d_int6h_spc1d_91x144.nc4
stats_global_GEOSFP_GEOSFP_MERRA2_20241201-
    20241231_len8d_int6h_spc1d_91x144.nc4
```

The stats.run script includes a warning that it is not recommended to process more than the equivalent of 31 init_dates with 10-day forecasts at 6 -hour spacing at once. However, you may be able to process significantly more by adjusting the time, memory, and chunking parameters in the “SBATCH Configuration Options” section of stats.run (you may also need to adjust these if processing an unusually high number of variables, levels, and/or regions).

The use of the stats.py arguments can be seen in how stats.py is called in the various jobs created by stats.run, for example:

Check for pre-existing datasets:

```
python -u stats.py --config ${versioned_yaml} --${dataset_type}
--collection ${collection} --info_dir ${sts_dir}
```

Non-chunked dataset-specific (collection-specific and non-collection-specific) processing jobs:

```
python -u stats.py --config ${versioned_yaml} --${dataset_type}
--collection ${collection} --info_dir ${sts_dir}
python -u stats.py --config ${versioned_yaml} --${dataset_type}
--info_dir ${sts_dir}
```

Chunked default collection forecasts processing and chunks merging jobs:

```
python -u stats.py --config ${versioned_yaml} --fcst ${collection_arg}
--date_start_idx ${chunk_start_idx} --date_end_idx ${chunk_end_idx}
--info_dir ${sts_dir}
python -u stats.py --config ${versioned_yaml} --merge_forecast_chunks
--info_dir ${sts_dir} ${merge_args}
```

Note \${collection_arg} and \${merge_arg} translate to “--save_for_coll_merge” if more collections than just default are being processed.

Collections merging job:

```
python -u stats.py --config ${versioned_yaml} --merge_collections
${dataset_type} --info_dir ${sts_dir}
```

Chunked regional and global stats jobs:

```
python -u stats.py --config '$versioned_yaml' --stats reg
--init_start_idx '$start_idx' --init_end_idx '$end_idx' --info_dir
'${sts_dir}' --chunk '$i' --chunk_size '$stats_chunk_size'
python -u stats.py --config '$versioned_yaml' --stats glo
--init_start_idx '$start_idx' --init_end_idx '$end_idx' --info_dir
'${sts_dir}' --chunk '$i' --chunk_size '$stats_chunk_size'
```

Regional and global stats merging jobs:

```
python -u stats.py --config ${versioned_yaml} --clean --info_dir  
${sts_dir} --type reg  
python -u stats.py --config ${versioned_yaml} --clean --info_dir  
${sts_dir} --type glo
```

PLOTS

The `plots_corcmp.py` and `plots_syscmp.py` scripts plot processed forecasts stats (from `stats.py`) to compare two or more models. The `plots_corcmp.py` script creates corcmp plots (dieoff curve and individual/comparison zonal mean) and montages for regional statistics data, while `plots_syscmp.py` creates static and animated plotsys (individual) and syscmp (comparison) systematic error plots. Together, they functionally replace dozens of GrADS scripts including the top-level `quickstat` script.

1. How plots_corcmp.py and plots_syscmp.py work

a. Input

The input is `stats_regional*.nc4` files (for corcmp) and `stats_global*.nc4` files (for syscmp) produced by `stats.py`. Files are required for a minimum of 2 models, one of which is designated the control model, and may be provided for up to 10. All requested dates, levels, variables, regions, etc., must be included for each model.

More than one file can be used for each model, with certain restrictions. For corcmp, there can be no overlap (no duplicate dates). For syscmp, there can be no overlap or extra (unrequested) dates. For both, all the files for a given experiment should have the same timing/resolution configuration (the part of the filename after the date range except date exclusions, i.e., “_len5d_int12h_spc1d_91x144.nc4”).

Among the models, for corcmp, each model’s file(s) must have equal or longer forecast length (“len”), equal or finer forecast interval (“int”), and equal or finer spacing (“spc”) than requested plotting parameters. For syscmp, length and interval have flexibility as with corcmp, but spacing and stats resolution (i.e., 91x144) must be identical among the models.

b. Output

The plots produced by `plots_corcmp.py` consist of (a) dieoff curves, (b) zonal means, and (c) montages. These plots are made for the following 6 stats: Anomaly Correlation, Root Mean Square (RMS) Error, RMS for Bias Error, RMS for Random Error, RMS for Amplitude Error, and RMS for Phase Error.

- a. Dieoff curve plots compare all models for a given stat/variable/level/region (see [Figure 1](#) for an example). The top panels display stat value vs. forecast lead day curves for each model with dashed lines showing the upper and lower 95% synoptic confidence intervals (see the “Confidence intervals” section below for explanation). The bottom panels show the difference between each model and the control model, with error bars/rectangles representing 68/90/95/99% confidence intervals from 2-tailed t-tests (when more than two models are plotted, only the 95% confidence intervals are shown).

- b. Zonal mean plots show the stats for all levels of a 3D variables vs forecast lead day. Plots are made for each model compared to the control model (see [Figure 2](#)) where colors denote magnitude of the difference and patterns denote significance from a 2-tailed t-test. Zonal mean plots are also made for each model by itself (simple line contours). Up to three varieties of each plot are made, depending on the minimum pressure level for which stats are available: linear y-axis up to 100-mb, log y-axis up to 10-mb, and log y-axis up to 1-mb.
- c. Montage plots come in 3 varieties. First, RMS decomposition montage plots are made showing all the RMS dieoff curves on one plot for each variable/level/region (see [Figure 3](#) for an example). Second, zonal mean comparison plots are combined into 2x3 grids to show 6 regions (SHE, TRO, NHE, SPO, XPO, and NPO) at once (see [Figure 4](#) for an example). Third, zonal mean comparison plots are combined into 4x5 grids to show 5 key variables (H, Q, U, T, and V) across 4 key regions (GLO, NHE, SHE, and TRO) as well as versions with their land-only equivalents (see [Figure 5](#) for an example).

The plots produced by `plots_syscmp.py` consist of global static/animated (a) individual and (b) comparison systematic error plots. For a given model/variable/level combo, static plots are produced for each requested lead time and then also saved together as an animated gif. For both individual and comparison plots, global map versions are made for each 2D variable or 3D variable/level combo, and zonal mean versions (including log variants as applicable) are made for each 3D variable.

- a. Individual model plots (see [Figure 6](#) for an example) have 8 panels (listed below left-to-right, top-to-bottom) with the global average included in the subplot titles:
 - Forecast: forecast variable values
 - Forecast-Climatology: difference between forecast and climatology variable values
 - Mean Error: the difference between the forecast and analysis variable values
 - Anomaly Correlation, Root Mean Square Error, Root Bias Error, Root Amplitude Error, and Root Phase Error: calculated statistics for the individual model
- b. Comparison model plots (see [Figure 7](#) for an example) have 6 panels (corresponding to all but the first two panels in the individual model plots) showing the difference in the stat for the comparison model minus the control model. The globally averaged differences are included in the subplot titles. The thin line contours denote areas of 90% significance from a 2-tailed t-test.

c. Confidence intervals

As noted in the explanation of dieoff curve plots, two different types of significance calculations are used for the confidence intervals (CIs): regular and synoptic 2-tailed t-tests. Additionally, transforms are applied to RMS and ACORR statistics before the significance calculations, resulting in intentionally asymmetric CIs. Note there are some slight math differences here from what was used in the old GrADS-based plotting routines which utilized the transforms and sample variance inconsistently.

The methodology for calculating the regular CIs is as follows. First, a transform is applied to both control and comparison model data. A Fisher transform (inverse hyperbolic tangent) is used for ACORR and a power transform (squaring the data) for all RMS stats. Then, differences and means are calculated in the transformed space. From these differences, variance and standard error are calculated and used to compute the t-critical value (from a two-sided t-test) and the resulting CIs. The CIs are added/subtracted to the transformed control mean then back-transformed into regular space. Finally, the back-transformed control mean is subtracted off to produce asymmetric CIs around zero (see the bottom panel in [Figure 1](#))

For the synoptic CIs, all models being plotted are considered, not just a control-comparison pair. As with the regular CIs, the appropriate transform is first applied to all data. Next, a control mean and variance are calculated in the transformed space. A variance is also calculated for each of the other models, but these are then averaged into a single comparison model variance. Then these two variances (control and comparison) are averaged together, and a standard error computed. From there, the process is similar to the regular scenario: the t-critical value is found and used to calculate CIs which are applied to the transformed control mean. The CIs are then back-transformed into regular space and the back-transformed control mean is subtracted off to isolate the asymmetric error margins. Finally, the original (untransformed) control mean is added to properly center the CIs around the control data (see the top panel in [Figure 1](#)).

d. Example plots

Figure 1: example dieoff curve plot

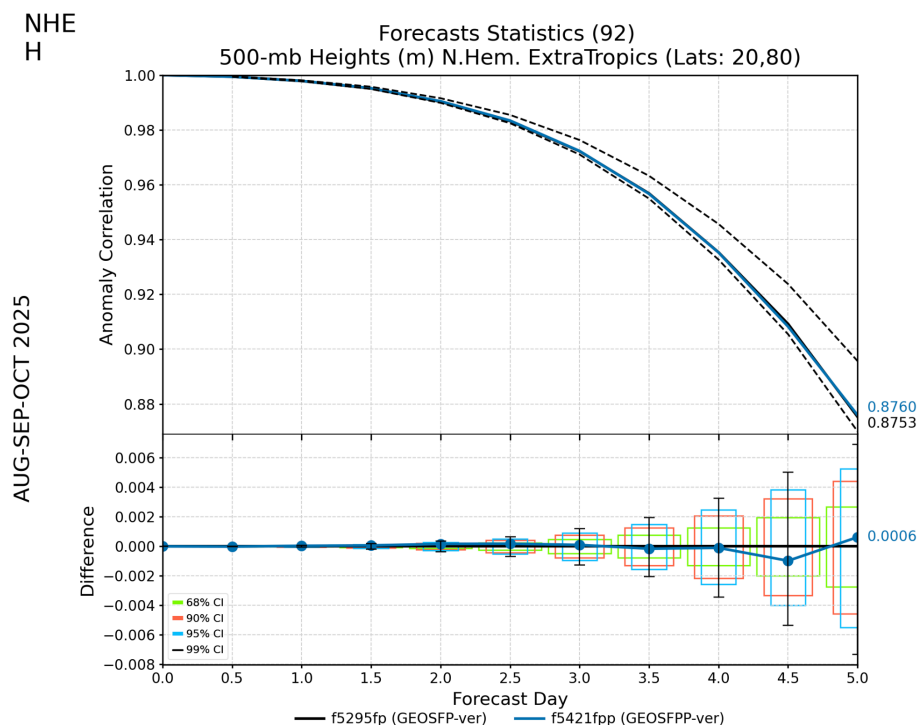


Figure 2: example zonal mean comparison plot (linear 100-mb)

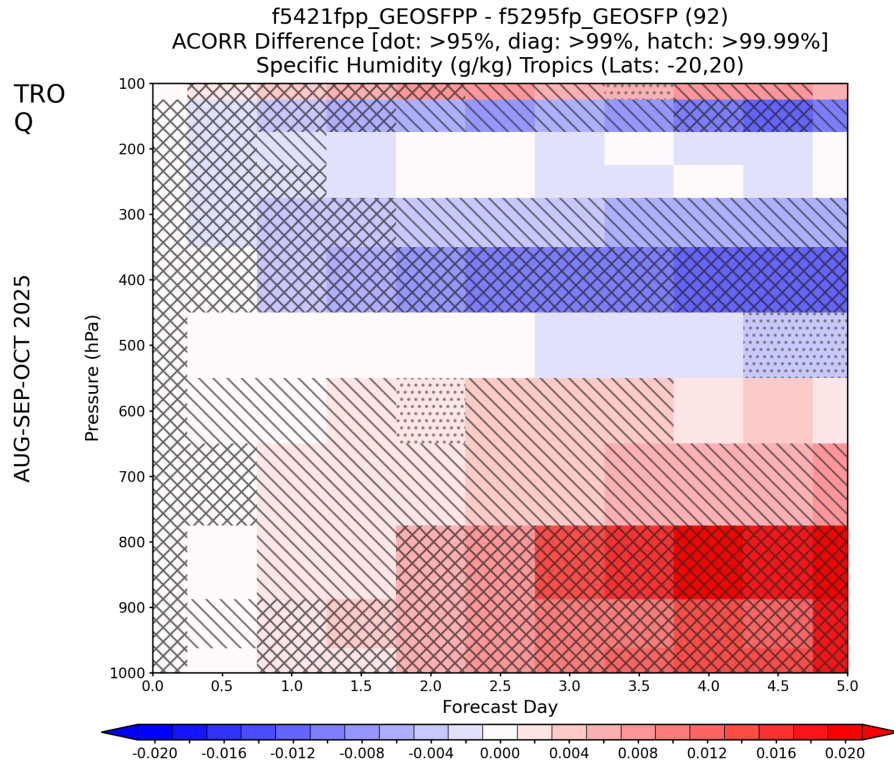


Figure 3: example RMS decomposition montage plot

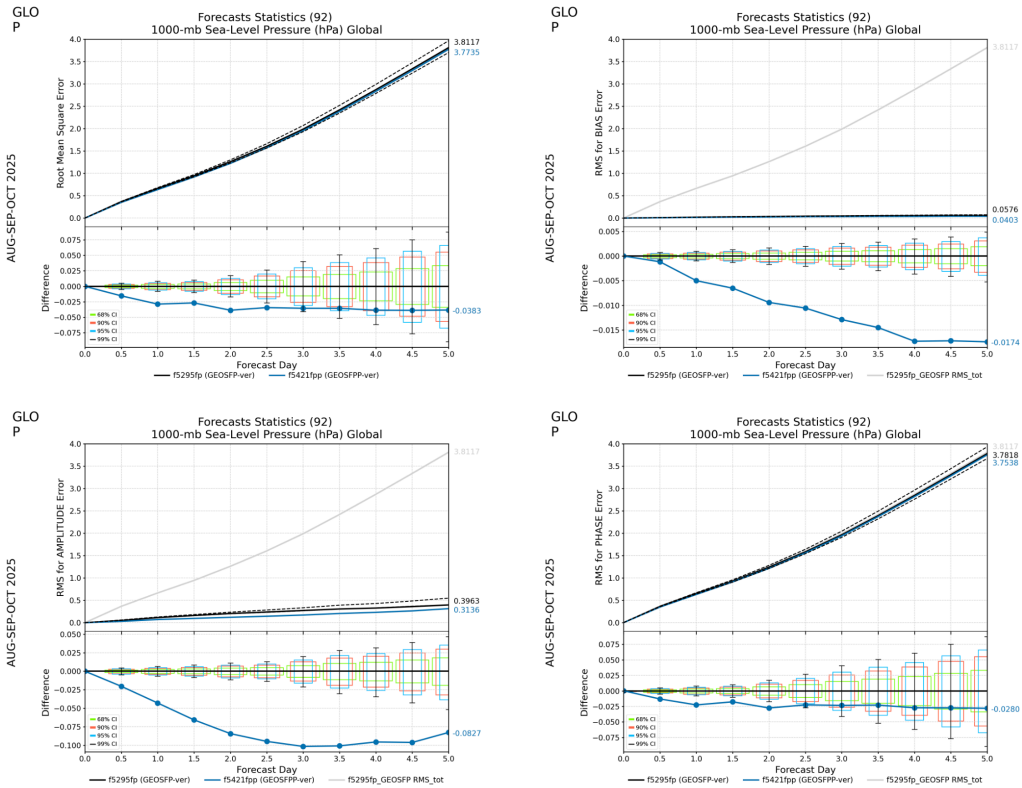


Figure 4: example regional montage plot (log 10-mb)

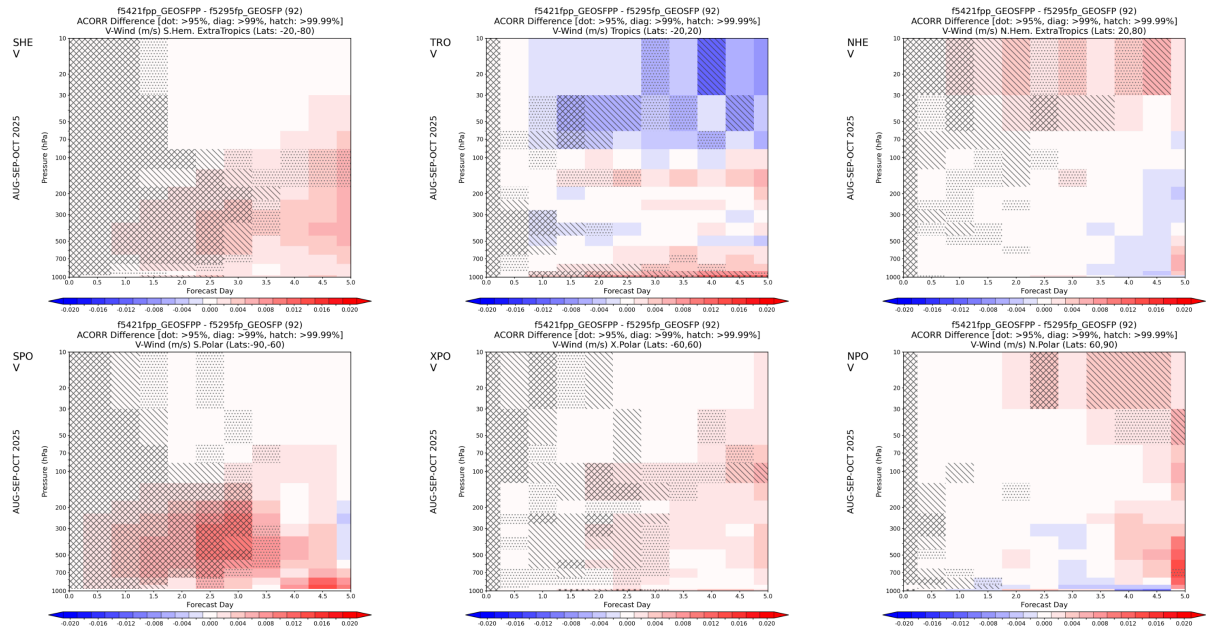


Figure 5: example overview stats montage plot (linear 10-mb; land-only)

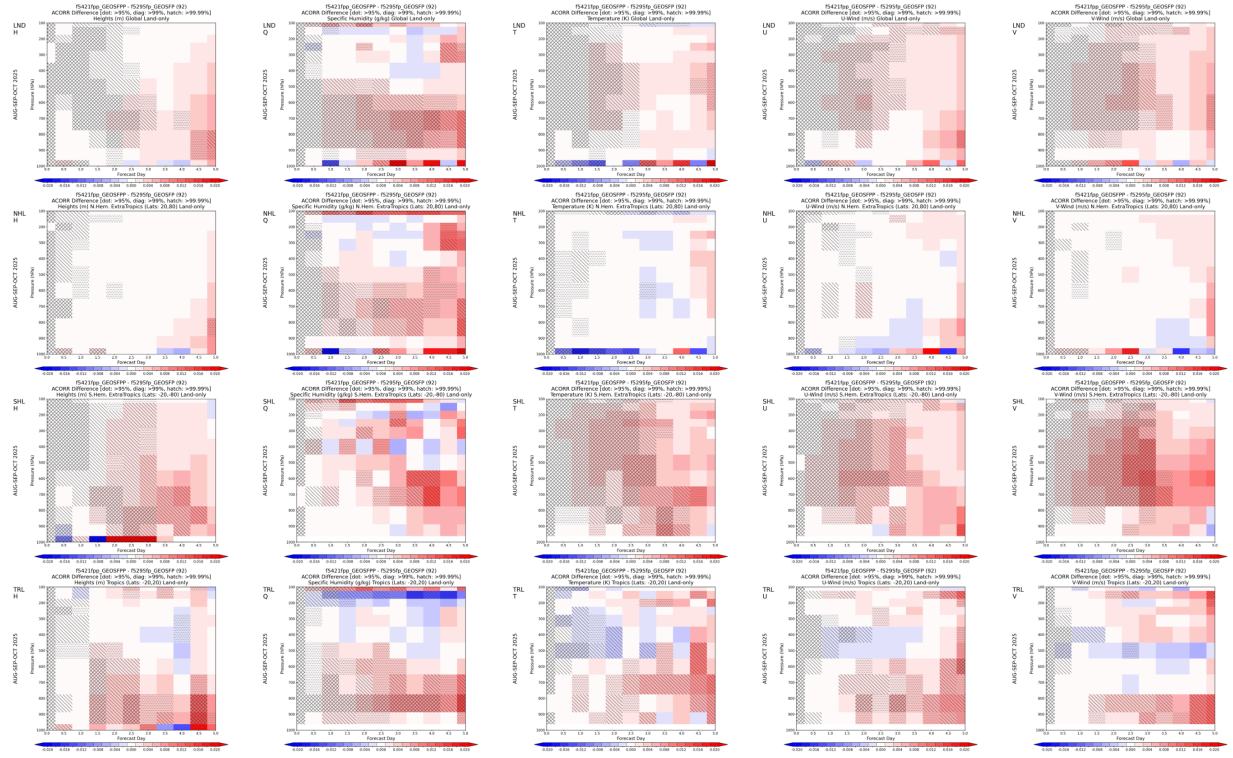


Figure 6: example global map individual static plot

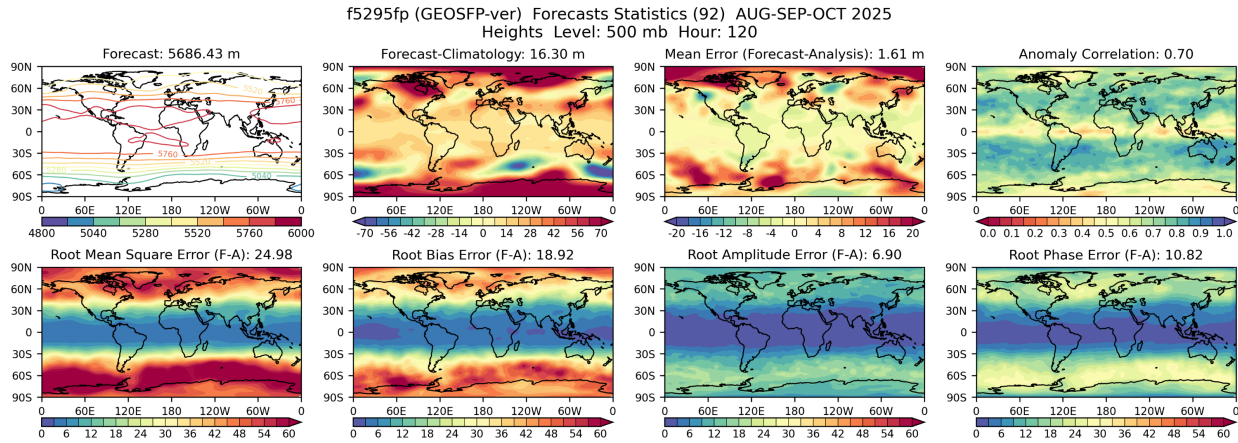
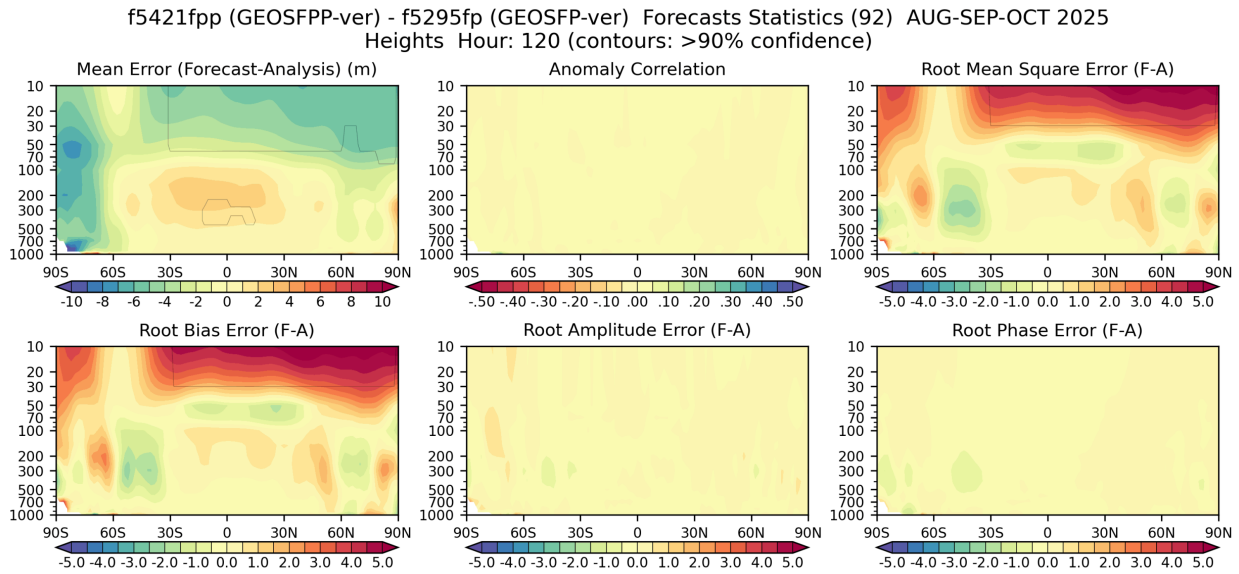


Figure 7: example global zonal mean comparison static plot (log 10-mb)



2. How to run plots_corcmp.py and plots_syscmp.py

a. Configuration

Running plots_corcmp.py or plots_syscmp.py requires a yaml configuration file. For a template, see plots_tmpl.yaml.

output:

The directory name suffix provides a unique name for the plots output directory. For example, if the suffix parameter is "FP_FPP", the output directory will be

“plots_FP_FPP”. The option to choose to make plots for corcmp, syscmp, or both is only used by plots.run, not plots_corcmp.py or plots_syscmp.py.

experiment stats files:

Regional stats files are required for corcmp plots and global stats files for syscmp plots. These should be specified using their full paths. The control experiment is “exp0”, and up to “exp9” is supported. Optionally, experiment forecast names can be overridden for plot legends (by default, the names in the stats files are used). Also, since corcmp zonal comparison plots only compare two models, additional requested comparison pairs can be listed. For example, with 3 experiments (exp0, exp1, and exp2), by default, zonal comparison plots would be made for exp0 vs. exp1 and exp0 vs. exp2, and it might be desirable to add [1, 2] (exp1 vs. exp2) to additional_pairs.

plotting selections:

As with stats.py, levels, variables, and regions should all be specified as comma-separated lists in brackets. Order within the list does not matter. Variables are separated by collections: default [H, Q, U, V, T, P, and PS], slices [2m and 10m variables], and aerosols. Default variables are separated by 2D and 3D. Regions are only required when making corcmp plots.

plotting options:

The season is used both in plot labels and filenames (for syncing with polar), while the year is only used in the plot labels. The plot_dpi option can be used to specify the output quality of the images. Setting “plot_RMS_decomp” to False will skip production of RMS decomposition plots (RMS_RANDOM, _BIAS, _AMPLITUDE, and _PHASE plots); traditional RMS and ACORR plots are always made. The colormap options allow the user to change the colormap used in filled contour plots from the defaults of bwr for corcmp and a custom spectral-like map for syscmp.

dates and times:

Forecast length/interval/spacing and start and end dates can be specified just as for stats.py. If not provided or blank, these parameters will be deduced from the control experiment.

SBATCH parameters:

These are only used when running plots_corcmp.py or plots_syscmp.py via the plots.run shell script as is discussed below.

b. Use via command line

Running plots_corcmp.py on the command line is simple:

```
plots_corcmp.py [--config CONFIG] [--process {plot,mont}]
```

where CONFIG is the yaml configuration file (can be specified relative to the working directory or as a full path), and process refers to creating all the regular plots ("plot") or the montages ("mont"). Thus, making all the corcmp plots requires two calls to plots_corcmp.py, the first to produce the regular plots and the second (after the first is complete) to create the montages from the previously made plots.

Similarly, the command line call for plots_syscmp.py is:

```
plots_syscmp.py [--config CONFIG] [--process {indiv,comp}]
```

where CONFIG is again the yaml configuration file, and process refers to creating individual ("indiv") or comparison ("comp") plots. Thus, as with the corcmp plots, making all the syscmp plots requires two calls to plots_syscmp.py, but they can be run in either order or simultaneously.

Additional arguments (see full lists below) are principally useful for parallelizing by splitting up the plotting among multiple jobs.

The command line arguments for plots_corcmp.py are as follows:

--config	CONFIG	Configuration file (default: plots.yaml)
--process	{plot,mont}	plot or mont
--part	PART	split up plots by region

The command line arguments for plots_syscmp.py are as follows:

--config	CONFIG	Configuration file (default: plots.yaml)
--process	{indiv,comp}	indiv or comp
--part	PART	split up plots by var
--exp	EXP	index of experiment

c. Use via plots.run

The plots.run shell script implements a comprehensive workflow to produce stats plots efficiently on the NCCS Discover system. It should be run in the command line with only a yaml file as an argument, for example: `./plots.run plots_FP_FPP.yaml`. It will look for the plots_corcmp.py and plots_syscmp.py scripts first in the working directory (WD) then in \$GEOSUTIL/plots_v2/stats_v2.

First, plots.run creates a "plots_{timestamp}" output directory (OD) in WD/output and saves a versioned copy of the referenced yaml configuration file to OD/jobs. Then it checks for the existence of the stats files specified in the yaml (only those needed; for example, if plots_type = corcmp, only the existence of the cor_exp files). It will then create (in OD/jobs) and submit jobs to the queue with appropriate dependencies (on other jobs) and account, partition, and qos as

included in the yaml. The stats files check and the job creation/submission will all be summarized in the command line output.

For corcmp plots, two jobs are submitted. The first is an array job splitting up the main plots by region, and the second is a follow-up jobs creating montage plots from the previously created plots. For syscmp plots, two array jobs are submitted, one for individual plots and one for comparison plots; these run simultaneously. Each syscmp array separates the jobs by both experiment and (each 3D var is separate and all 2D vars are together). Note the comparison jobs run a lot slower than all the other plots due to calculation of difference significance for every lat/lon grid box.

The plots are organized under WD/output/plots with the same structure as in the old GrADS-based system: a corcmp directory, a syscmp directory for each individual model and comparison pair, and a corcmp.rc file to facilitate uploading of plots to Polar. The plots are named the same as before, except that the experiment name now includes both forecast and analysis (hyphenated so it still works with polar) to facilitate comparisons of the same forecast to different analyses.

If memory/time limits cause issues, these can be adjusted in the “SBATCH Configuration Options” section of plots.run.

d. Usage of plots_syscmp.py: need for cartopy

Since syscmp plots include coastlines, plots_syscmp.py checks for the presence of cartopy coastline data. If not found, execution is halted and an instructional message is displayed to the user:

*This script requires coastline data that must be downloaded first.
To fix this, please run the following command ON A LOGIN NODE:*

```
python -c 'import matplotlib.pyplot as plt; import cartopy.crs as ccrs;
fig = plt.figure(); ax = plt.axes(projection=ccrs.PlateCarree());
ax.coastlines(); plt.savefig("test.png"); import os;
os.remove("test.png"); print("Map data downloaded successfully")'
```

*This downloads ~2MB of coastline data to ~/.local/share/cartopy/
You only need to do this once, then you can run this script.*

e. Syncing with Polar

To facilitate syncing with Polar (for displaying on the GMAO GEOS-5 Model Development page) a “corcmp.rc” text file is created in the top-level plots directory whenever plots_corcmp.py is run. This is a stripped-down version as it is only used for syncing with Polar (while previously corcmp.rc was also used with the plotting routines). It simply lists the names of the experiments (as used in the plot filenames), for example:

DSC0: f5295fp-GEOSFP
DSC1: f5421fpp-GEOSFPP

Then, movestat can be run just like previously, for example:

```
$GEOSUTIL/plots/movestat -expid f5295fp -year Y2025 -rc `pwd`/corcmp.rc
```

On Polar, several php files (in /san_agcm/geos5/[expid]/) should be edited for the plots to display correctly on the “Forecast Statistics” page. For the following files in the “buttons” directory, the needed change is to comment out and/or create name pair(s) as (web directory name, display name), for example:

```
buttons/stats_year.button:  
    set_item ( $year,"Y2025_SELF","Y2025 (SELF-ver)" );
```

```
buttons/stats_season.button:  
    set_item ( $season , "DJF" , "December-January-February" );
```

```
buttons/stats_exps.php:  
    set_item($cmpexp, "-----SELF-ver-----", "-----SELF-ver-----");  
    set_item($cmpexp, "f5295fp-GEOSFP", "GEOSFP (FP-ver)");  
    set_item($cmpexp, "f5421fpp-GEOSFPP", "GEOSFPP (FPP-ver)");  
    set_item($cmpexp, "-----ERA5-ver-----", "-----ERA5-ver-----");  
    set_item($cmpexp, "f5295fp-ERA5", "GEOSFP (ERA5-ver)");  
    set_item($cmpexp, "f5421fpp-ERA5", "GEOSFPP (ERA5-ver)");
```

For index.php, simply edit the “\$stats_args” line to hard code year, season, exp1, and exp2 for page defaults, for example:

```
$stats_args =  
"&year=Y2025_SELF&season=DJF&region=GL0&type=corcmp&expid=$expid&cmpexp=  
=1&exp1=f5295fp-GEOSFP&exp2=f5421fpp-GEOSFPP&gifs=Animated&verbose=0";
```